Object Oriented Programming

Mihai Dascălu

Methods

Problem

```
int sum = 0;
for (int i = 1; i <= 10; i++)
  sum += i;
System.out.println("Sum from 1 to 10 is " + sum);

sum = 0;
for (int i = 20; i <= 30; i++)
  sum += i;
System.out.println("Sum from 20 to 30 is " + sum);

sum = 0;
for (int i = 35; i <= 45; i++)
  sum += i;
System.out.println("Sum from 35 to 45 is " + sum);
```

Solution

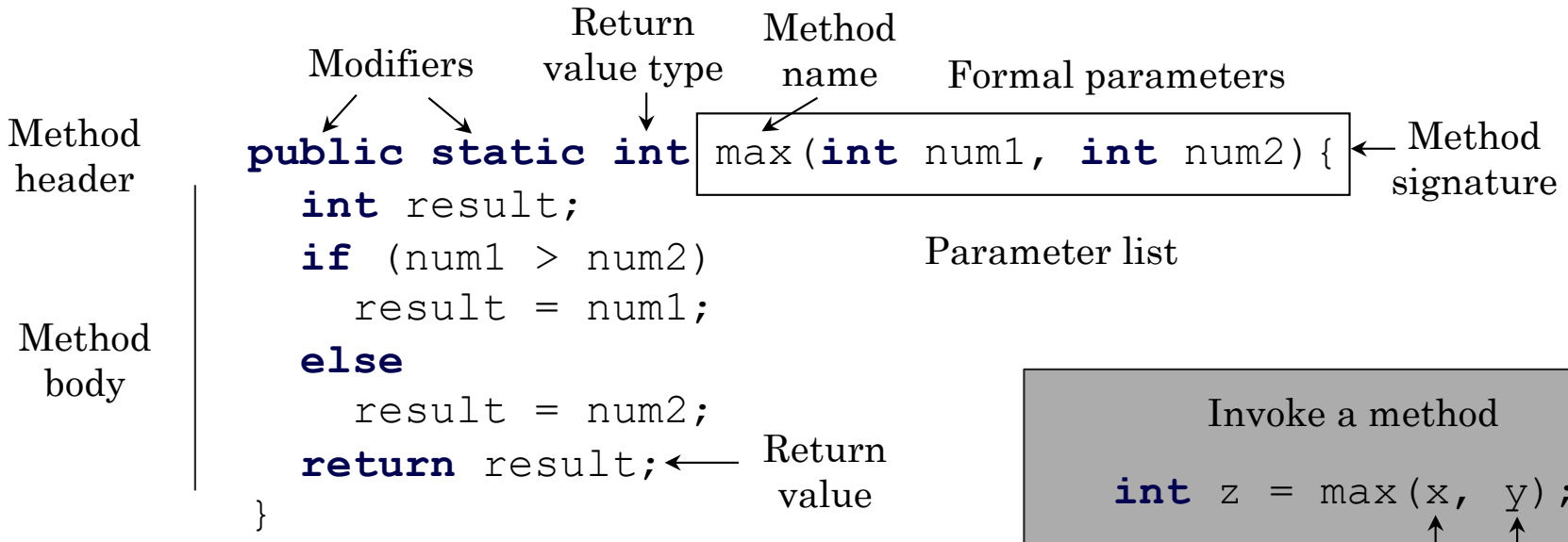
```
public static int sum(int i1, int i2) {
  int sum = 0;
  for (int i = i1; i <= i2; i++)
    sum += i;
  return sum;
}
```

```
public static void main(String[] args) {
  System.out.println("Sum from 1 to 10 is " +
sum(1, 10));
  System.out.println("Sum from 20 to 30 is " +
sum(20, 30));
  System.out.println("Sum from 35 to 45 is " +
sum(35, 45));
}
```

Defining Methods (1)

- A collection of statements grouped together to perform specific operations

Define a method



Invoke a method

```
int z = max(x, y);
```

actual parameters (arguments)

Defining Methods (2)

- Signature = combination of method name + parameter list
- Formal Parameters = variables defined in the method header
- Actual parameter or argument = when invoking a method is, a **value is passed to the parameter**
- A method may return a value (otherwise **void**)
- The returnValueType = data type of the value the method returns

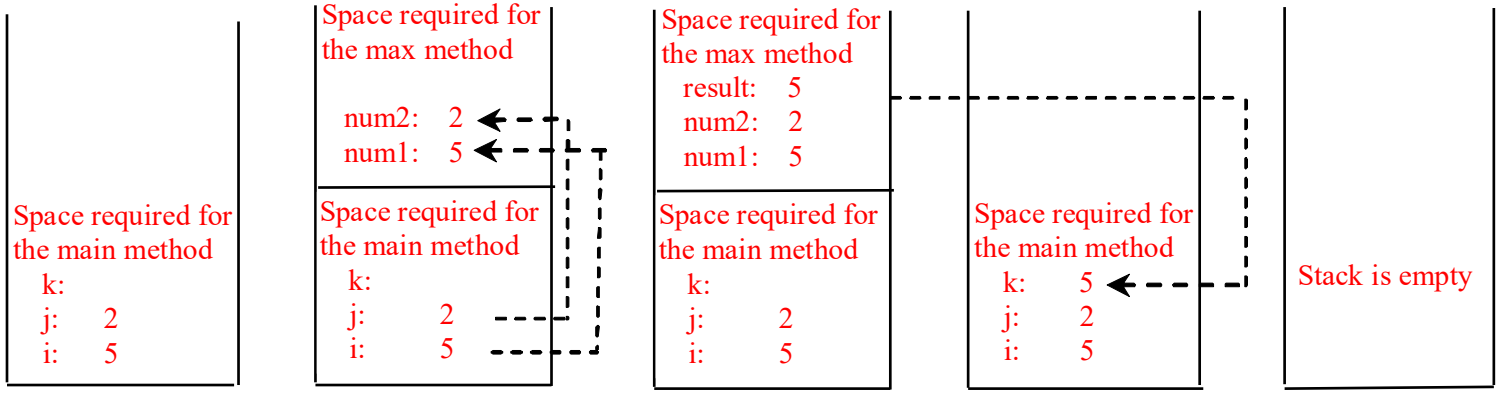
Calling Methods & Call stack

pass the value of i & j

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;
  if (num1 > num2)
    result = num1;
  else
    result = num2;
  return result;
}
```



(a) The main method is invoked.

(b) The max method is invoked.

(c) The max method is being executed.

(d) The max method is finished and the return value is sent to k.

(e) The main method is finished.

CAUTION

- A return statement is required for a value-returning method
- Method (a) is logically correct, but it has a compilation error
  - the Java compiler thinks it possible that this method does not return any value
- Fix: delete *if (n < 0)* so that the compiler will see a return statement to be reached regardless of how if is evaluated

```
public static int sign(int n) {
  if (n > 0)
    return 1;
  else if (n == 0)
    return 0;
  else if (n < 0)
    return -1;
}
```

```
public static int sign(int n) {
  if (n > 0)
    return 1;
  else if (n == 0)
    return 0;
  else
    return -1;
}
```

Passing Parameters

- Suppose you invoke the method using

```
nPrintln("Welcome to Java", 5);
```

- What is the output?
- Suppose you invoke the method using

```
nPrintln("Computer Science", 15);
```

- What is the output?

```
public static void nPrintln(String message, int n) {
  for (int i = 0; i < n; i++)
    System.out.println(message);
}
```

Overloading Methods

Overloading the previous `max` method

```
public static double max(double num1, double num2) {
  if (num1 > num2)
    return num1;
  else
    return num2;
}
```

Ambiguous Invocation

- Sometimes there may be two or more possible matches for an invocation of a method

=>

- The compiler cannot determine the most specific match (ambiguous invocation)

=>

Compilation error

Ambiguous Invocation

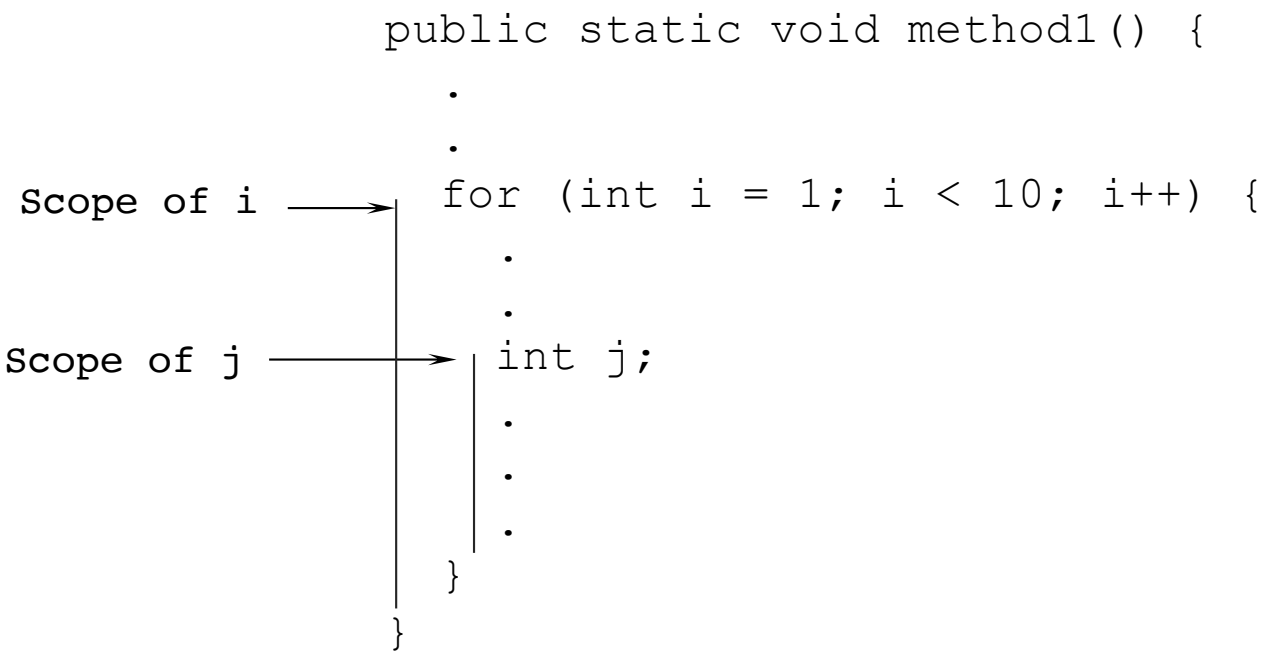
```
public class AmbiguousOverloading {
   public static void main(String[] args){
     System.out.println(max(1, 2));
   }
   public static double max(int num1, double num2) {
     if (num1 > num2)
       return num1;
     else
       return num2;
   }
   public static double max(double num1, int num2) {
     if (num1 > num2)
       return num1;
     else
       return num2;
   }
}
```

Scope of Local Variables

- **A local variable**: a variable defined inside a method
- **Scope**: the part of the program where the variable can be referenced
  - Starts from its declaration and continues to the end of the block that contains the variable
  - A local variable must be declared before it can be used
- You can declare a local variable with the same name multiple times in different non-nesting blocks in a method
- You *cannot* declare a local variable twice in nested blocks

Scope of Local Variables, cont.

- A variable declared in the initial action part of a for loop header has its scope in the entire loop
- A variable declared inside a *for* loop body has its scope limited in the loop body



Scope of Local Variables, cont.

```
//Fine


public static void method1() {
  int x = 1;
  int y = 1;
  for (int i = 1; i < 10; i++) {
    x += i;
  }
  for (int i = 1; i < 10; i++) {
    y += i;
  }
}
```

```
//Wrong to declare i in two
nesting blocks

public static void method2() {
  int i = 1;
  int sum = 0;
  for (int i = 1; i < 10; i++) {
    sum += i;
  }
}
```

Scope of Local Variables, cont.

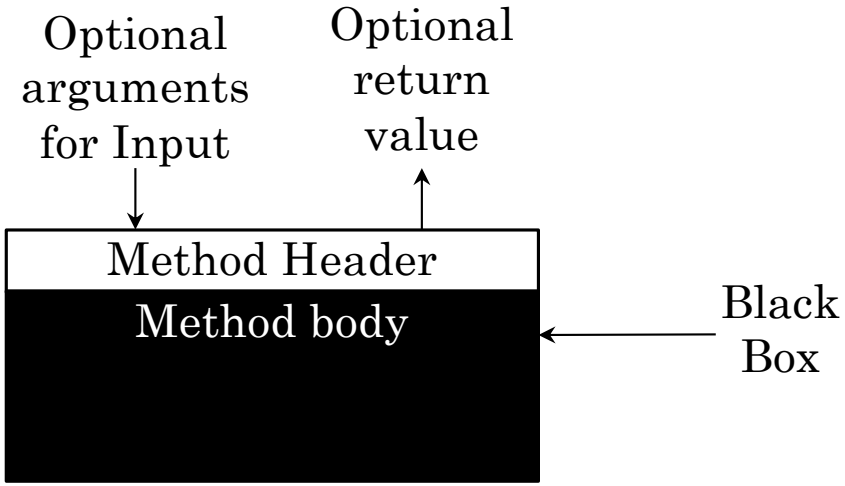
```
// Fine with no errors
public static void correctMethod() {
  int x = 1;
  int y = 1;
  // i is declared
  for (int i = 1; i < 10; i++) {
    x += i;
  }
  // i is declared again
  for (int i = 1; i < 10; i++) {
    y += i;
  }
}
```

Scope of Local Variables, cont.

```
// With errors
public static void incorrectMethod(int x) {
  int x = 1;
  int y = 1;
  for (int i = 1; i < 10; i++) {
    int x = 0;
    x += i;
  }
}
```

Benefits of Methods

- Write a method once and reuse it anywhere
- Modularity of code & application improvement
- Information hiding – hide the implementation from the user
- Reduce complexity
- Abstraction



Stepwise Refinement

- Method abstraction can be applied to the process of developing applications
- When writing a large program, you can use the "divide and conquer" strategy

  = stepwise refinement

  Decompose it into subproblems, further decomposed into smaller, more manageable problems
- Recommendation (≈20-40 lines of code)

Implementation: Top-Down

- Implement one method in the structure chart at a time from the top to the bottom
- **Stubs** (a simple, but incomplete version of a method) for the methods waiting to be implemented
- Enables programmers to test the method invocation from a caller
  - e.g., implement the main method first and then use a stub for the printMonth method

Implementation: Bottom-Up

- Implement one method in the structure chart at a time from the bottom to the top
- For each implemented method, write a test program to test it
- Both top-down and bottom-up methods are applicable and useful
- Both approaches implement the methods incrementally and help to isolate programming errors and makes debugging easy
- Sometimes, an integrated approach can be used

The `Math` Class

- Class constants:
  - `PI`
  - `E`
- Class methods:
  - Trigonometric Methods
  - Exponent Methods
  - Rounding Methods
  - min, max, abs, and random Methods

Trigonometric Methods

- `sin(double a)`
- `cos(double a)`
- `tan(double a)`
- `acos(double a)`
- `asin(double a)`
- `atan(double a)`

Radians

toRadians(90)

```
Examples:

Math.sin(0) returns 0.0
Math.sin(Math.PI / 6) returns 0.5
Math.sin(Math.PI / 2) returns 1.0
Math.cos(0) returns 1.0
Math.cos(Math.PI / 2) returns 0
```

Exponent Methods

- `exp(double a)`
  Returns `e` raised to the power of `a`.
- `log(double a)`
  Returns the natural logarithm of `a`.
- `log10(double a)`
  Returns the 10-based logarithm of `a`.
- `pow(double a, double b)`
  Returns `a` raised to the power of `b`.
- `sqrt(double a)`
  Returns the square root of `a`.

```
Examples:


Math.exp(1) returns 2.71
Math.log(2.71) returns 1.0
Math.pow(2, 3) returns 8.0
Math.pow(3, 2) returns 9.0
Math.pow(3.5, 2.5) returns
    22.91765
Math.sqrt(4) returns 2.0
Math.sqrt(10.5) returns 3.24
```

Rounding Methods

- `double ceil(double x)`

  x rounded up to its nearest integer. This integer is returned as a double value.

- `double floor(double x)`

  x is rounded down to its nearest integer. This integer is returned as a double value.

- `double rint(double x)`

  x is rounded to its nearest integer. If x is equally close to two integers, the even one is returned as a double.

- `int round(float x)`

  Return (int)Math.floor(x+0.5).

- `long round(double x)`

  Return (long)Math.floor(x+0.5).

min, max, and abs

- `max(a, b)` and `min(a, b)`

  Returns the maximum or minimum of two parameters.

- `abs(a)`

  Returns the absolute value of the parameter.

- `random()`

  Returns a random `double` value
  in the range [0.0, 1.0).

```
Examples:


Math.max(2, 3) returns 3
Math.max(2.5, 3) returns 3.0
Math.min(2.5, 3.6) returns
   2.5
Math.abs(-2) returns 2
Math.abs(-2.1) returns 2.1
```

The random Method

- Generates a random double value greater than or equal to 0.0 and less than 1.0

(0 <= Math.random() < 1.0)

```
(int)(Math.random() * 10)
```

Returns a random integer between 0 and 9.

```
50 + (int)(Math.random() * 50)
```

Returns a random integer between 50 and 99.

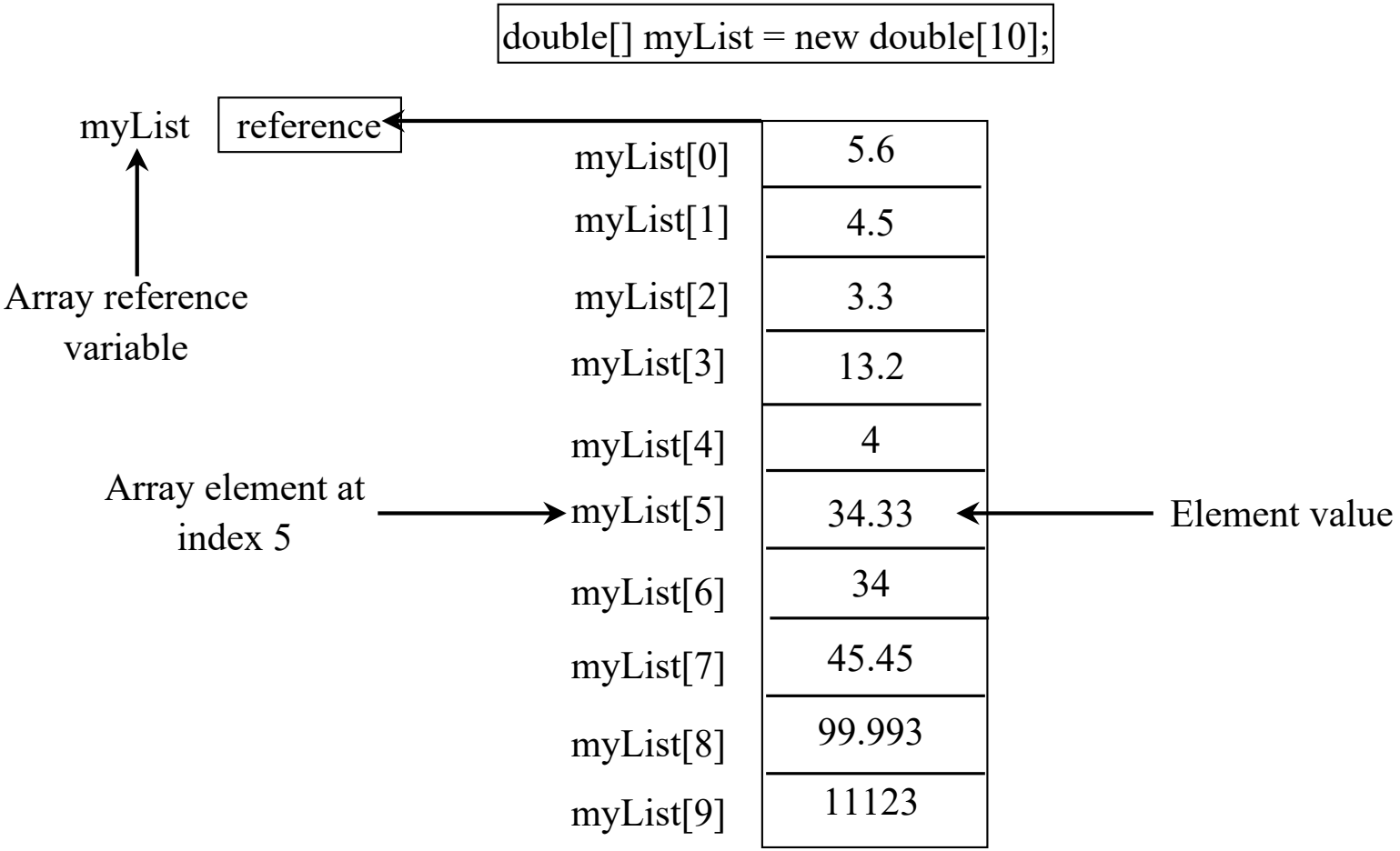
```
a + Math.random() * b
```

Returns a random number between a and a + b, excluding a + b.

Arrays

Introducing Arrays

- Data structures that represent collections of the same types of data



Declaring Array Variables

- `datatype[] arrayRefVar;`

  Example:

  ```
  double[] myList;
  ```

- `datatype arrayRefVar[];`

  // This style is allowed, but not preferred

  Example:

  ```
  double myList[];
  ```

Creating Arrays

```
arrayRefVar = new datatype[arraySize];
```

Example:

```
myList = new double[10];
```

`myList[0]` references the first element in the array

`myList[9]` references the last element in the array

Declaring and Creating in One Step

- ```
  datatype[] arrayRefVar = new
      datatype[arraySize];
  ```

  ```
  double[] myList = new double[10];
  ```

- ```
  datatype arrayRefVar[] = new
    datatype[arraySize];
  ```

  ```
  double myList[] = new double[10];
  ```

The Length of an Array

- Once an array is created, its size is fixed
- It cannot be changed ☹
- You can find its size using `arrayRefVar.length`
  - myList.length returns 10

Default Values

When an array is created, its elements are assigned the default value of

- 0 for the numeric primitive data types
- '\u0000' for char types
- false for boolean types
- null for objects

Indexed Variables

- Accessed through the index - *0-based* indices (0 to arrayRefVar.length-1)
- Each element in the array is represented using the following syntax, known as an *indexed variable*: arrayRefVar[index]
- After an array is created, an indexed variable can be used in the same way as a regular variable

```
myList[2] = myList[0] + myList[1];
```

Array Initializers

- Declaring, creating, initializing in one step:

```
double[] myList = {1.9, 2.9, 3.4, 3.5};
```

- This shorthand syntax must be in one statement
  - Splitting it causes a syntax error

```
double[] myList;
myList = {1.9, 2.9, 3.4, 3.5};
```



Wrong!

Shorthand Notation

```
double[] myList = {1.9, 2.9, 3.4, 3.5};
```

Shorthand notation equivalent to the following:

```
double[] myList = new double[4];
myList[0] = 1.9;
myList[1] = 2.9;
myList[2] = 3.4;
myList[3] = 3.5;
```

Trace it!

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

Initializing Arrays

```
java.util.Scanner input = new
    java.util.Scanner(System.in);

System.out.print("Enter " + myList.length + "
    values: ");

for (int i = 0; i < myList.length; i++)
  myList[i] = input.nextDouble();
```

```
for (int i = 0; i < myList.length; i++) {
  myList[i] = Math.random() * 100;
}
```

```
for (int i = 0; i < myList.length; i++) {
  System.out.print(myList[i] + " ");
}
```

Summing Elements & Finding the Max Value

```
double total = 0;
for (int i = 0; i < myList.length; i++) {
  total += myList[i];
}
```

```
double max = myList[0];
for (int i = 1; i < myList.length; i++) {
  if (myList[i] > max) max = myList[i];
}
```

Enhanced for Loop (for-each loop)

- JDK 1.5+: a new for loop that enables traversing an array sequentially without using an index variable:

```
for (double value: myList)
  System.out.println(value);
```

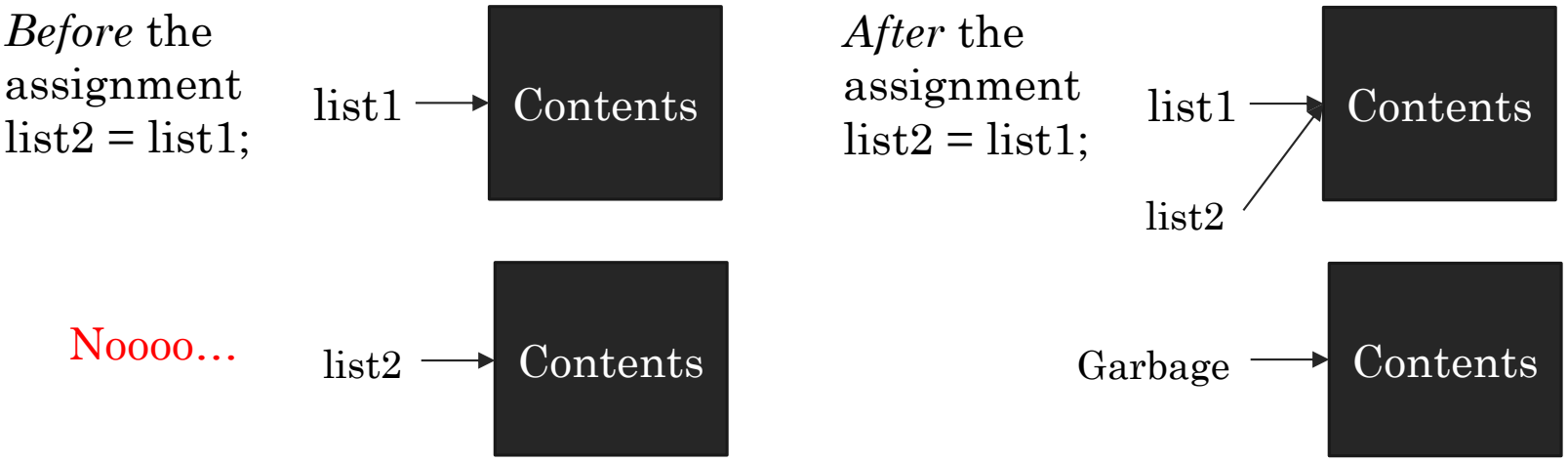
- General syntax:

```
for (elementType value: arrayRefVar) {
  // Process the value
}
```

- An index variable needs to be used if you wish to traverse the array in a different order or change the elements

Copying Arrays

- Often, in a program, you need to duplicate an array or a part of an array
- In such cases you could attempt to use the assignment statement (=), as follows:



Copying Arrays

```
//Using a loop:
int[] sourceArray = {2, 3, 1, 5, 10};
int[] targetArray = new int[sourceArray.length];


for (int i = 0; i < sourceArrays.length; i++){
   targetArray[i] = sourceArray[i];
}
```

The arraycopy Utility

```
arraycopy(sourceArray, src_pos, targetArray,
 tar_pos, length);
```

Example:

```
System.arraycopy(sourceArray, 0,
  targetArray, 0, sourceArray.length);
```

Anonymous Array

- The statement

```
new int[]{3, 1, 2, 6, 4, 2};
```

creates an array using the following syntax:

```
new dataType[] {literal0, literal1, ...,
 literalk};
```

- There is no explicit reference variable for the array => *anonymous array*

Passing Arrays to Methods

```
public static void printArray(int[] array) {
  for (int i = 0; i < array.length; i++) {
    System.out.print(array[i] + " ");
  }
}
```

```
//Invoke the method
int[] list = {3, 1, 2, 6, 4, 2};
printArray(list);
```

```
//Invoke the method
printArray(new int[]{3, 1, 2, 6, 4, 2});
```

Anonymous array

Pass By Value

- Java uses ***pass by value*** to transfer arguments to a method
- There are **important** differences in terms of parameters:
  - For a *primitive type value*, the *actual value* is passed
    - Changing the value of the local parameter inside the method ***does not*** *affect* the value
  - For an *array type*, the value of the parameter contains a *reference* to an array => this reference is passed to the method
    - Any changes to the array that occur inside the method body ***will*** *affect* the original array

Swap Example (1)

```
public static void main(String[] args) {
  int[] a = {1, 2};
  // Swap elements using the swap method
  swap(a[0], a[1]);
  // Swap elements using the swapFirstTwoInArray method
  swapFirstTwoInArray(a);
}
/** Swap two variables */
public static void swap(int n1, int n2) {
  int temp = n1;
  n1 = n2;
  n2 = temp;
}
/** Swap the first two elements in the array */
public static void swapFirstTwoInArray(int[] array) {
  int temp = array[0];
  array[0] = array[1];
  array[1] = temp;
}
```

Trace Returning an Array from a Method

```
int[] list1 = new int[]{1, 2, 3, 4, 5, 6};

int[] list2 = reverse(list1);


public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    result[j] = list[i];
  }

  return result;
}
```

Searching Arrays

- Look for a specific element in an array (e.g., discovering whether a certain score is included in a list of scores)
  - *Linear search*
  - *Binary search*

```
public class LinearSearch {
  /** The method for finding a key in the list */
  public static int linearSearch(int[] list, int key) {
    for (int i = 0; i < list.length; i++)
      if (key == list[i])
        return i;
    return -1;
  }
}
```

Binary Search

```
/** Use binary search to find the key in the list */
public static int binarySearch(int[] list, int key) {
  int low = 0;
  int high = list.length - 1;
  while (high >= low) {
    int mid = (low + high) / 2;
    if (key < list[mid])
      high = mid - 1;
    else if (key == list[mid])
      return mid;
    else
      low = mid + 1;
  }
   return -1 - low;
}
```

The Arrays.binarySearch Method

- Java provides several overloaded binarySearch methods:

```
int[] list = {2, 4, 7, 10, 11, 45, 50, 59, 60,
  66, 69, 70, 79};
System.out.println("Index is " +
  java.util.Arrays.binarySearch(list, 11));
```

Return is 4

```
char[] chars = {'a', 'c', 'g', 'x', 'y', 'z'};
System.out.println("Index is " +
  java.util.Arrays.binarySearch(chars, 't'));
```

Return is -4 (insertion point is 3, so return is -3-1)

- For the binarySearch method to work, the array must be pre-sorted in increasing order.

Selection Sort

- Sorting, like searching, is also a common task
- Many different algorithms have been developed for sorting
- Selection sort:
  - Find the largest number in the list and places it last
  - Finds the largest remaining number and places it next to last
  - Until the list contains only one number

Wrap it in a Method

```
/** The method for sorting the numbers */
public static void selectionSort(double[] list) {
   for (int i = 0; i < list.length; i++) {
      // Find the minimum in the list[i..list.length-1]
      double currentMin = list[i];
      int currentMinIndex = i;
      for (int j = i + 1; j < list.length; j++) {
        if (currentMin > list[j]) {
          currentMin = list[j];
          currentMinIndex = j;
        }
      }
      // Swap list[i] with list[currentMinIndex], if needed;
      if (currentMinIndex != i) {
        list[currentMinIndex] = list[i];
        list[i] = currentMin;
      }
   }
}
```

Invoke it

```
selectionSort(yourList)
```

The Arrays.sort Method

- Java provides several overloaded sort methods in the java.util.Arrays class

```
double[] numbers = {6.0, 4.4, 1.9, 2.9, 3.4, 3.5};
java.util.Arrays.sort(numbers);

char[] chars = {'a', 'A', '4', 'F', 'D', 'P'};
java.util.Arrays.sort(chars);
```

Multidimensional Arrays

Declare/Create Two-dimensional Arrays

- Two-dimensional arrays can be used to represent a matrix or a table

```
// Declare array ref var

dataType[][] refVar;

// Create array and assign its reference to variable

refVar = new dataType[10][10];

// Combine declaration and creation in one statement

dataType[][] refVar = new dataType[10][10];

// Alternative syntax

dataType refVar[][] = new dataType[10][10];
```

Declaring Variables of Two-dimensional Arrays

```
int[][] matrix = new int[10][10];
```

or

```
int matrix[][] = new int[10][10];

matrix[0][0] = 3;

for (int i = 0; i < matrix.length; i++)
  for (int j = 0; j < matrix[i].length; j++)
    matrix[i][j] = (int)(Math.random() * 1000);

double[][] x;
```

Declaring, Creating, and Initializing using Shorthand Notations

- An array initializer can be used to declare, create, and initialize a two-dimensional array

```
int[][] array = {
  {1, 2, 3},
  {4, 5, 6},
  {7, 8, 9},
  {10, 11, 12}
};
```

Same as

```
int[][] array = new int[4][3];
array[0][0] = 1; array[0][1] = 2; array[0][2] = 3;
array[1][0] = 4; array[1][1] = 5; array[1][2] = 6;
array[2][0] = 7; array[2][1] = 8; array[2][2] = 9;
array[3][0] =10; array[3][1] =11; array[3][2] =12;
```

Lengths of Two-dimensional Arrays, cont.

```
int[][] array = {
  {1, 2, 3},
  {4, 5, 6},
  {7, 8, 9},
  {10, 11, 12}
};
```

```
array.length
array[0].length
array[1].length
array[2].length
array[3].length
```

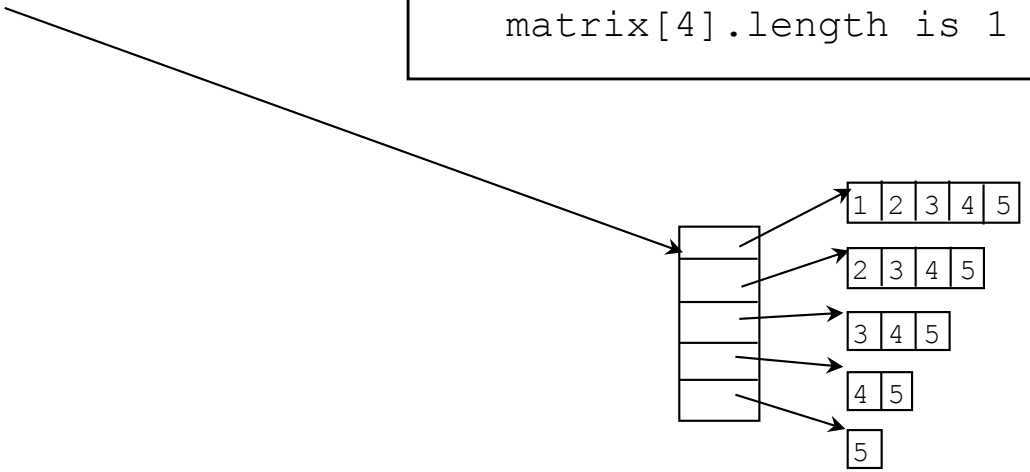
```
array[4].length    ArrayIndexOutOfBoundsException
```

Ragged Arrays

- Each row in a two-dimensional array is itself an array
- Rows can have different lengths » *ragged array*

```
int[][] matrix = {
   {1, 2, 3, 4, 5},
   {2, 3, 4, 5},
   {3, 4, 5},
   {4, 5},
   {5}
};
```

```
matrix.length is 5
matrix[0].length is 5
matrix[1].length is 4
matrix[2].length is 3
matrix[3].length is 2
matrix[4].length is 1
```



Initializing/printing arrays with random values

```
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length;
     column++) {
    matrix[row][column] = (int)(Math.random() * 100);
  }
}
```

```
for (int[] row :matrix) {
  for (int el : row)
    System.out.print(el+"\t");
  System.out.println();
}
```

Random shuffling

```
for (int i = 0; i < matrix.length; i++) {
  for (int j = 0; j < matrix[i].length; j++) {
    int i1 = (int)(Math.random() * matrix.length);
    int j1 = (int)(Math.random() * matrix[i].length);
    // Swap matrix[i][j] with matrix[i1][j1]
    int temp = matrix[i][j];
    matrix[i][j] = matrix[i1][j1];
    matrix[i1][j1] = temp;
  }
}
```

Multidimensional Arrays

- Occasionally, the is a need to represent n-dimensional data structures
  - To be avoided where possible in OOP
- Create n-dimensional arrays for any integer *n* in Java
- Generalize to declare n-dimensional array variables for n >= 3

```
double[][][] scores = new double[10][5][2];
```